Бие даалт №1 — Хэсэгчлэх ба дахин дуудах арга

Алгоритмын зохиомж ба шинжилгээ (F.CSM301)

А түвшин · Python, Java

Оюутан: Б. Билгүүн (**B190910014**)

Багш: Д. Батмөнх

ШУТИС, МХТС · Улаанбаатар, 2026

1. Ерөнхий мэдээлэл

1.1 Ажлын зорилго

Хэсэгчлэх аргаар бодлогыг дэд бодлогод хувааж, дэд бодлогыг дахин дуудалтаар бодож, шийдүүдийг нэгтгэх зарчмыг хэрэгжүүлэх. А түвшний шаардлагын дагуу таван бодлогыг бүгдийг **Python** болон **Java** дээр бодож, 4 ба 5-р бодлогын гүйцэтгэлийг **сайжруулав**. Бодлого бүрийн хугацаа, санах ойн зарцуулалтыг онолоор тооцоолж, бодит хэмжилтээр шалгав.

№	Бодлого	Түвшин	Арга
1	Majority Element (LeetCode 169)	C	хэсэгчлэх
2	Convert Sorted Array to Binary Search Tree (LeetCode 108)	C	дахин дуудах
3	Longest Nice Substring (LeetCode 1763)	C	хэсэгчлэх
4	Different Ways to Add Parentheses (LeetCode 241)	B → A	хэсэгчлэх → товъёг + санах (memo)
5	Construct Binary Tree from Preorder and Inorder Traversal (LeetCode 105)	B → A	дахин дуудах → хэш хүснэгт + товъёг

1.2 Хэсэгчлэх арга ба дахин дуудах тэгшитгэл

Хэсэгчлэх арга гурван алхамтай: **хуваах** (ижил төрлийн жижиг дэд бодлогууд), **бодох** (дэд бодлогыг дахин дуудалтаар; хангалттай жижиг бол шууд), **нэгтгэх** (дэд шийдүүдээс эх бодлогын шийд). n хэмжээтэй бодлогыг n/b хэмжээтэй a дэд бодлогод хувааж, хуваах ба нэгтгэхэд f(n) ажил зарцуулбал

T(n) = a·T(n/b) + f(n).

Ерөнхий теоремоор (лекцийн Теорем 5.1–5.2) f(n)-г n^(log_b a) урсгал функцтэй харьцуулна: (1) f = O(n^(log_b a − ϵ)) → Θ(n^(log_b a)); (2) f = Θ(n^(log_b a) lg^k n), k ≥ 0 → Θ(n^(log_b a) lg^(k+1) n); (3) f = Ω(n^(log_b a + ϵ)) ба a·f(n/b) ≤ c·f(n), c < 1 → Θ(f(n)). Тэгш биш хуваалттай (жишээ нь 5-р бодлого) тэгшитгэлийг давталтын мод эсвэл орлуулгаар бодно.

1.3 Файлын бүтэц

```
prj1/B190910014/
├── python/src/      p1_majority_element.py … p5_build_tree_improved.py, tree_node.py
├── python/tests/    test_problems.py           (pytest)
├── python/bench.py                             хэмжилт
├── java/src/main/java/com/prj1/   MajorityElement.java … BuildTreeImproved.java, Bench.java
├── java/src/test/java/com/prj1/   ProblemsTest.java (JUnit 5)
├── report/          prj1_report.md/.pdf, make_tables.py, build_pdf.py
└── env_info.sh
```

LeetCode-д илгээхдээ Python-д `class Solution`-ийг шууд, Java-д класын нэрийг `Solution` болгож (`package` мөрийг хасаж) илгээнэ. `TreeNode`-ийг LeetCode өөрөө өгдөг.

1.4 Тестийн ерөнхий үр дүн

Бодлого бүрийг LeetCode-ийн жишээ болон **шууд аргатай (brute force) харьцуулсан санамсаргүй тохиолдлоор** шалгав:

Бодлого	Шалгасан зүйл
1	300 санамсаргүй массив, `Counter` / `groupingBy` -ээр тоолсон хариутай
2	n = 0…1000: inorder нь эрэмбэлэгдсэн (хайлтын мод), тэнцвэртэй, өндөр яг ⌈lg(n+1)⌉
3	2000 санамсаргүй мөр, бүх дэд мөрийг шалгах $\Theta(n^3)$ аргатай
4	k = 0…8: анхны ба сайжруулсан хариу ижил, хариуны тоо = Каталаны тоо C_k, дуудалт 3^k ба $(k+1)(k+2)/2$
5	200 санамсаргүй мод + 2000 зангилаатай гинж: сэргээсэн модны preorder, inorder анхныхтай ижил

Хэл	Команд	Үр дүн
Python	`cd python && .venv/bin/pytest -v`	28 passed
Java	`cd java && mvn test`	Tests run: 6, Failures: 0

1.5 Хэмжилтийн орчин ба арга

WSL2 Ubuntu 22.04, Intel Core i5-1334U (12 логик цөм, L2 7.5 MiB, L3 12 MiB), 7.6 GiB RAM, Python 3.10.12, OpenJDK 17.0.20.1 (`env_info.sh`).

- **Хугацаа.** Нэг багц ≥ 20 ms байтал дуудалтыг давтаж, нэг дуудалтын дундажийг гаргав. Python-д 3, Java-д 5 багцын медиан. Java-д эхний багцуудаар JIT-ийг халааж, хэмжилт бүрийн өмнө GC ажиллуулав.
- **Санах ой.** Python: `tracemalloc` -ийн **оргил** утга (дуудалтын явцад Python-ы объектуудад нэгэн зэрэг эзэлсэн хамгийн их санах ой). Java: `ThreadMXBean.getThreadAllocatedBytes` -ээр нэг дуудалтад **нийт хуваарилсан** санах ой (GC-ээр чөлөөлөгдсөн ч тоологдоно). Хоёр хэмжигдэхүүн ижил биш тул хэл хооронд шууд харьцуулахгүй, харин n-ээр өсөх хандлагыг харьцуулна. Дахин дуудалтын стек аль алинд нь тоологдохгүй.

Хэмжилтийг дахин хийх: `python3 bench.py`, `java -Xss64m -cp build com.prj1.Bench` → `out/results_<lang>.tsv` → `python report/make_tables.py`.

2. Бодлого 1. Majority Element (LeetCode 169)

2.1 Зорилго

n урттай массиваас ⌊n/2⌋-аас олон удаа орсон (олонх) гишүүнийг олно. Ийм гишүүн заавал байна.

2.2 Алгоритмын санаа

Ажиглалт: m нь бүх массивын олонх гишүүн бол зүүн эсвэл баруун хагасын **ядаж нэгд нь** олонх байна. Хэрэв хоёр хагаст хоёуланд нь хагасаас олон биш байсан бол нийт тоо нь n/2-оос хэтрэхгүй. Тиймээс: 1. **Хуваах:** [lo, hi)-г дундуур нь хоёр хуваана. 2. **Бодох:** хагас бүрийн олонх гишүүнийг дахин дуудалтаар олно (суурь: ганц гишүүн). 3. **Нэгтгэх:** хоёр нэр дэвшигч ижил бол тэр. Өөр бол [lo, hi)-д хоёуланг тоолж, олон удаа орсныг сонгоно.

```
def majority(lo, hi):                # nums[lo:hi]-ийн олонх гишүүн
    if hi - lo == 1:
        return nums[lo]
    mid = (lo + hi) // 2
    left, right = majority(lo, mid), majority(mid, hi)
    if left == right:
        return left
    left_count = sum(1 for i in range(lo, hi) if nums[i] == left)
    right_count = sum(1 for i in range(lo, hi) if nums[i] == right)
    return left if left_count > right_count else right
```

2.3 Хугацаа

Хамгийн муу үед түвшин бүрт тоолох ажил Θ(n): **T(n) = 2T(n/2) + Θ(n)** → a = b = 2, n^($\log_2 2$) = n = f(n), 2-р тохиолдол (k = 0) → **Θ(n lg n)**. Хамгийн сайн үед (бүх гишүүн ижил) нэгтгэх алхам Θ(1) тул T(n) = 2T(n/2) + Θ(1) = Θ(n).

n	Python, ms	Java, ms	Python t / (n lg n), нс	Python санах ой, KB	Java санах ой, KB
1,000	1.126	0.012	113	1.6	0.0
10,000	12.8	0.118	96	2.2	0.0
100,000	123.6	1.572	74	2.5	0.0
1,000,000	1,431	16.0	72	2.8	0.0

n 10 дахин өсөхөд хугацаа ≈ 10·lg(10n)/lg n дахин өсөх ёстой. Python-д 10^5 → 10^6 үед 11.6 дахин (онол 12.0), Java-д 10.2 дахин өссөн. «t / (n lg n)» багана 113 → 72 нс гэж бага зэрэг буурч, том n дээр тогтворжиж байна. Жижиг n дээр функцийн дуудалт, хэмжилтийн тогтмол зардал харьцангуй их байдаг тул ингэж буурч байгаа нь Θ(n lg n)-тэй зөрөхгүй.

2.4 Санах ой

Массивыг хуулахгүй, зөвхөн (lo, hi) товъёгоор дамжуулна. Нэмэлт санах ой нь дахин дуудалтын стекийн гүн ⌈lg n⌉ буюу **Θ(lg n)**. Хэмжилтэд Python-ы оргил хэдхэн KB, Java-д хуваарилалт 0 байгаа нь (стек тоологдохгүй) үүнийг харуулна.

Харьцуулахад Boyer–Moore-ийн санал хураах арга нэг удаа гүйгээд **Θ(n)** хугацаа, **Θ(1)** санах ойтой. Хэсэгчлэх арга lg n дахин удаан ч бодлогыг хувааж, нэгтгэх санааг тодорхой харуулдаг.

3. Бодлого 2. Convert Sorted Array to Binary Search Tree (LeetCode 108)

3.1 Зорилго

Өсөхөөр эрэмбэлэгдсэн массиваас **өндрөөр тэнцвэртэй** хоёртын хайлтын мод байгуулна.

3.2 Алгоритмын санаа

Дунд гишүүнийг үндэс болговол зүүн тал нь бүгд бага, баруун тал нь бүгд их (хайлтын модны нөхцөл), хоёр талын хэмжээ хамгийн ихдээ 1-ээр ялгаатай (тэнцвэр). Хоёр талыг дахин дуудалтаар байгуулна.

```
def build(lo, hi):                    # nums[lo..hi]
    if lo > hi:
        return None
    mid = (lo + hi) // 2
    return TreeNode(nums[mid], build(lo, mid - 1), build(mid + 1, hi))
```

Модны өндөр яг ⌈lg(n+1)⌉ бөгөөд энэ нь n зангилаатай хоёртын модны хамгийн бага өндөр (тестээр шалгав).

3.3 Хугацаа

Дуудалт бүр Θ(1) ажил хийж хоёр хагаст дуудна: **T(n) = 2T(n/2) + Θ(1)** → n^(log₂2) = n ≫ 1 = O(n^(1−ϵ)), ϵ = 1, 1-р тохиолдол → **Θ(n)**. Зангилаа бүрийг яг нэг удаа үүсгэдэг тул Ω(n)-ээс бага байж чадахгүй.

n	Python, ms	Java, ms	Python санах ой, KB	Java санах ой, KB	Java, байт/зангилаа
1,000	0.662	0.040	143.9	23.5	24
10,000	12.4	0.116	1,480	234.4	24
100,000	187.8	1.015	14,840	2,344	24
1,000,000	6,825	42.7	148,435	23,438	24

Java-д n = 10^4 → 10^5 үед хугацаа 8.7 дахин өссөн. 10^5 → 10^6 үед 42 дахин өссөн нь 23 MB шинэ объект үүсгэхэд GC ажилласантай холбоотой. Python-д 10^6 зангилаа үүсгэхэд санах ой хуваарилалт, GC давамгайлж шугамаас хурдан өссөн. Гэхдээ хийж буй **алхмын тоо** яг n дуудалт тул онолын эрэмбэ Θ(n) хэвээр.

3.4 Санах ой

Гаралт өөрөө n зангилаа тул **Θ(n)**, үүнээс гадна дахин дуудалтын стек Θ(lg n). Java-д зангилаа бүр яг 24 байт (объектын толгой 12 + int 4 + хоёр заагч 2×4, 8-д тэгшилсэн) эзэлж байгаа нь хүснэгтийн сүүлийн баганаас харагдана. Python-ы объект илүү том (≈ 150 байт/зангилаа).

4. Бодлого 3. Longest Nice Substring (LeetCode 1763)

4.1 Зорилго

Үсэг бүр нь том, жижиг хоёр хэлбэрээрээ орсон мөрийг «сайхан» гэнэ. Хамгийн урт сайхан дэд мөрийг олно. Ижил урттай хэд байвал хамгийн эхнийхийг буцаана.

4.2 Алгоритмын санаа

«Муу» үсэг: эсрэг хэлбэр нь s-д огт байхгүй үсэг. Муу үсэг агуулсан дэд мөр хэзээ ч сайхан болохгүй. Тиймээс: 1. **Хуваах:** s-ийг бүх муу үсгээр нь нэг дор хэсэглэнэ. 2. **Бодох:** хэсэг бүрийг дахин дуудалтаар бодно. 3. **Нэгтгэх:** хамгийн урт хариуг авна (тэнцүү бол эхнийх).

Муу үсэг байхгүй бол s өөрөө сайхан.

```
def longestNiceSubstring(s):
    if len(s) < 2:
        return ""
    present = set(s)
    best, start, split = "", 0, False
    for i, ch in enumerate(s):
        if ch.swapcase() not in present:          # муу үсэг
            split = True
            part = longestNiceSubstring(s[start:i])
            if len(part) > len(best):
                best = part
            start = i + 1
    if not split:
        return s
    part = longestNiceSubstring(s[start:])
    return part if len(part) > len(best) else best
```

4.3 Хугацаа

Хэсгүүдийн нийт урт $< n$ тул түвшин бүрийн нийт ажил $\Theta(n)$: $T(n) = \Sigma\, T(n_i) + \Theta(n)$, $\Sigma\, n_i < n$. **Гүний хязгаар:** нэг түвшинд хуваасан муу үсгийн төрөл хүүхдийн бүх хэсгээс бүрмөсөн алга болно. Тиймээс түвшин бүрт хэсэгт байгаа үсгийн төрлийн тоо дор хаяж 1-ээр буурч, гүн $\le |\Sigma| = 52$ болно. Нийт **$\Theta(|\Sigma| \cdot n) = \Theta(n)$** ($|\Sigma|$ тогтмол).

Хэрэв зөвхөн эхний муу үсгээр хуваавал «аааа…а» мэт оролтод гүн n болж $\Theta(n^2)$ болно. Бүх муу үсгээр нэг дор хуваах нь энэ муу тохиолдлоос сэргийлдэг.

n	Оролт	Python, ms	Java, ms	Python t / n, нс	Java t / n, нс	Python санах ой, KB	Java санах ой, KB
100	nice	0.019	0.001	190	10	0.9	0.2
100	split	0.081	0.008	810	80	3.6	3.5
1,000	nice	0.232	0.004	232	4	1.0	0.2
1,000	split	0.741	0.029	741	29	3.7	27.6
10,000	nice	2.193	0.084	219	8	1.0	0.2
10,000	split	8.143	0.800	814	80	4.4	307.4
100,000	nice	35.0	1.310	350	13	1.0	0.2
100,000	split	131.5	6.169	1315	62	4.6	2,902

«nice» оролтод мөр бүхэлдээ сайхан тул ганц удаа гүйж дуусна. «split» оролтод хос хэлбэргүй «z» үсгээр олон хуваалт хийгдэж, дахин дуудалт ажиллана. Аль алинд нь «t / n» тогтмол орчим байгаа нь $\Theta(n)$-ийг баталж байна.

4.4 Санах ой

`present` олонлог $\Theta(|\Sigma|)$. Python хувилбар дэд мөрийг хуулдаг тул нэг түвшинд $\Theta(n)$ түр санах ой, стекийн гүн ≤ 52. Java хувилбар хуулалгүй (lo, hi) товъёгоор дамжуулж, дуудалт бүрд `boolean[128]` хэрэглэдэг. Тиймээс **нэгэн зэрэг амьд** байх нэмэлт санах ой нь $\Theta(|\Sigma| \cdot \text{гүн}) = \mathbf{\Theta(1)}$ (гаралтын мөрөөс бусдаар). Python-ы оргил утга n-ээс үл хамааран ≈ 1–5 KB байгаа нь үүнийг харуулж байна.

Java-ийн хүснэгтэд «split» үед санах ой n-тэй хамт өсөж байгаа нь **нийт хуваарилалт** юм. Дуудалт бүр шинэ `boolean[128]` (≈ 144 байт) үүсгэх бөгөөд «z» үсэг 11 тэмдэгт тутамд нэг таарч, дуудалтын тоо ≈ n/11 болно: n = 10^5 үед ≈ 9 000 дуудалт × 144 байт + бусад ≈ 2.9 MB. Эдгээр массив дуудалт дуусмагц хог болдог тул оргил санах ойд нөлөөлөхгүй. «Нийт хуваарилалт» ба «оргил санах ой» гэдэг хоёр өөр хэмжигдэхүүнийг ялгах нь чухал.

5. Бодлого 4. Different Ways to Add Parentheses (LeetCode 241)

5.1 Зорилго

`+`, `-`, `*` үйлдэл ба тоо агуулсан илэрхийлэлд хаалтыг бүх боломжит аргаар тавьж гарах бүх утгыг буцаана.

5.2 Алгоритмын санаа (B түвшин)

Хамгийн сүүлд хийгдэх үйлдлийг сонгоод илэрхийллийг хоёр хуваана. Зүүн, баруун талын бүх утгыг дахин дуудалтаар олоод, хос бүрт тэр үйлдлийг хэрэглэнэ.

```
def diffWaysToCompute(expression):
    results = []
    for i, ch in enumerate(expression):
        if ch in "+-*":
            left = diffWaysToCompute(expression[:i])
            right = diffWaysToCompute(expression[i + 1:])
            for x in left:
                for y in right:
                    results.append(apply(x, y, ch))
    if not results:                       # үйлдэлгүй — ганц тоо
        results.append(int(expression))
    return results
```

5.3 Хугацаа ба санах ой (B түвшин)

k үйлдэлтэй илэрхийлэлд хариуны тоо нь **Каталаны тоо** $C_k = (2k)! / ((k+1)!\cdot k!) \approx 4^k / (k^{1.5}\cdot\sqrt{\pi})$. Гаралт өөрөө ийм хэмжээтэй тул аль ч алгоритм $\Omega(C_k)$.

Дуудалтын тоо: $C(0) = 1$, $C(k) = 1 + \Sigma_{j=0}^{k-1} [C(j) + C(k-1-j)] = 1 + 2\cdot\Sigma_{j=0}^{k-1} C(j)$ тул $C(k) = 3\cdot C(k-1)$ буюу **$C(k) = 3^k$** (тестээр шалгав). Гэтэл өөр өөр дэд илэрхийлэл ердөө $(k+1)(k+2)/2$ л бий. Тэгэхээр **ижил дэд бодлогыг олон дахин** бодож, дэд мөрийг хуулж, тоог дахин задалж байна. Түр жагсаалтуудын нийт хэмжээ ч гурвын зэргээр өснө.

5.4 Сайжруулсан шийдэл (A түвшин)

1. Илэрхийллийг **нэг удаа** тоо, үйлдлийн жагсаалт болгон задална (дэд мөр хуулахгүй).
2. Дэд бодлогыг операндын **товъёгийн муж (i, j)**-ээр тодорхойлж, хариуг **санана (memo)**.

```
def compute(i, j):                        # nums[i..j] операндууд
    if (i, j) in memo:
        return memo[(i, j)]
    if i == j:
        result = [nums[i]]
    else:
        result = [apply(x, y, ops[k])
                  for k in range(i, j)
                  for x in compute(i, k)
                  for y in compute(k + 1, j)]
    memo[(i, j)] = result
    return result
```

Дэд бодлого бүр яг нэг удаа бодогдоно: **(k+1)(k+2)/2** удаа. Хугацаа нь бүх дэд бодлогын хариуг байгуулах ажилтай тэнцүү, ойролцоогоор Θ(Σ_m (k+1−m)·C_m) = Θ(C_k). Энэ нь гаралтын хэмжээтэй ижил эрэмбэтэй тул асимптот оновчтой.

k	Хариу (Каталан)	Дуудалт: анхны	Дуудалт: сайжруулсан	Python анхны, ms	Python сайжр., ms	Java анхны, ms	Java сайжр., ms	Python анхны / сайжр.
2	2	9	6	0.012	0.017	0.002	0.004	0.7×
3	5	27	10	0.038	0.034	0.005	0.003	1.1×
4	14	81	15	0.089	0.033	0.014	0.003	2.7×
5	42	243	21	0.377	0.145	0.017	0.002	2.6×
6	132	729	28	0.978	0.139	0.044	0.005	7.0×
7	429	2,187	36	1.985	0.324	0.142	0.016	6.1×
8	1,430	6,561	45	7.464	0.982	0.456	0.046	7.6×
9	4,862	19,683	55	17.8	3.654	1.418	0.193	4.9×
10	16,796	59,049	66	59.9	9.266	4.369	0.498	6.5×

Дуудалтын тоо 3^k-аас (k+1)(k+2)/2 болж буурсан (k = 10: 59 049 → 66). Хугацаа k = 10 үед Python-д **6.5 дахин**, Java-д **8.8 дахин** хурдан болов. Хугацааны хурдсалт дуудалтын тооны харьцаа шиг (895 дахин) их биш. Учир нь хоёр хувилбарын аль аль нь C_k хэмжээтэй гаралтыг байгуулах ёстой бөгөөд энэ ажил сайжруулсан хувилбарын хугацааны ихэнхийг эзэлдэг. Сайжруулалт **давхардсан** ажлыг арилгасан, харин **гаралтын** ажлыг арилгах боломжгүй.

Санах ой. memo нь бүх (i, j) мужийн хариуг зэрэг хадгалдаг тул Python-ы **оргил** санах ой сайжруулсан хувилбарт бага зэрэг их байна (k = 10: 460 KB → 619 KB). Энэ бол хугацаа ба санах ойн солилцоо (time–space tradeoff). Харин Java-д нэг дуудалтад **нийт хуваарилсан** санах ой анхны хувилбарт 3^k-аар өсөж, сайжруулсанд хэд дахин бага байна:

k	Python анхны, KB	Python сайжр., KB	Java анхны, KB	Java сайжр., KB
2	0.7	1.5	1.4	1.4
3	1.0	2.1	4.4	2.2
4	1.7	3.5	13.3	2.2
5	2.6	5.4	32.7	4.2
6	7.0	14.3	103.4	10.8
7	20.4	37.1	318.3	30.2
8	67.1	111.4	982.8	94.1
9	225.6	342.9	3,060	307.2
10	460.4	618.5	9,415	1,025

6. Бодлого 5. Construct Binary Tree from Preorder and Inorder Traversal (LeetCode 105)

6.1 Зорилго

Ялгаатай утгатай хоёртын модны preorder ба inorder тойрлоос модыг сэргээнэ.

6.2 Алгоритмын санаа (B түвшин)

preorder[0] нь үндэс. inorder-т үндсээс зүүн талд зүүн дэд мод, баруун талд баруун дэд мод байрлана. Хэрэв үндэс inorder-ийн k байрлалд байвал preorder[1..k] нь зүүн дэд модны preorder. Хоёр жагсаалтыг хуваагаад хоёр дэд модыг дахин дуудалтаар сэргээнэ.

```
def buildTree(preorder, inorder):
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    k = inorder.index(preorder[0])                    # O(n) хайлт
    root.left = buildTree(preorder[1:k + 1], inorder[:k])
    root.right = buildTree(preorder[k + 1:], inorder[k + 1:])
    return root
```

6.3 Хугацаа ба санах ой (B түвшин)

Дуудалт бүр шугаман хайлт ба хуулалтад Θ(n) зарцуулна: T(n) = T(k) + T(n−1−k) + Θ(n). - **Тэнцвэртэй ойролцоо мод** (k ≈ n/2): T(n) = 2T(n/2) + Θ(n) = **Θ(n lg n)** (R1). - **Нэг талдаа тулсан мод** (k = n−1 буюу гинж): T(n) = T(n−1) + Θ(n) = Σ Θ(i) = **$\Theta(n^2)$**.

Санах ой: зүсмэл бүр шинэ жагсаалт. Гинжин модонд дахин дуудалтын зам дээрх бүх зүсмэл зэрэг амьд байх тул оргил санах ой n + (n−1) + … ≈ $n^2/2$ буюу **$\Theta(n^2)$** болно.

6.4 Сайжруулсан шийдэл (A түвшин)

1. inorder-ийн **утга** → **товъёг**-ийг хэш хүснэгтэд нэг удаа хадгална: хайлт **Θ(1)**.
2. Жагсаалт хуулахгүй, inorder-ийн (lo, hi) мужаар дамжуулна. preorder-ийг нэг заагчаар дараалаар уншина, учир нь preorder-т үндэс → зүүн дэд мод → баруун дэд мод гэсэн дараалалтай.

```
position = {value: i for i, value in enumerate(inorder)}
next_root = 0

def build(lo, hi):                                      # inorder[lo..hi]
    nonlocal next_root
    if lo > hi:
        return None
    value = preorder[next_root]; next_root += 1
    k = position[value]                                 # Θ(1)
    root = TreeNode(value)
    root.left = build(lo, k - 1)
    root.right = build(k + 1, hi)
    return root
```

Одоо T(n) = T(k) + T(n−1−k) + Θ(1) тул модны хэлбэрээс үл хамааран **Θ(n)**. Санах ой: хэш хүснэгт Θ(n) + стек Θ(h), h ≤ n.

Санамсаргүй мод:

n	Python анхны, ms	Python сайжр., ms	Java анхны, ms	Java сайжр., ms	Python анхны, KB	Python сайжр., KB	Java анхны, KB	Java сайжр., KB
500	1.567	0.592	0.108	0.542	87.5	94.5	81.2	56.9
1,000	1.678	1.222	0.066	0.053	172.0	200.4	169.1	115.7
2,000	4.927	2.308	0.132	0.183	328.6	418.4	364.2	233.3
4,000	6.605	5.714	0.278	0.380	613.5	835.9	760.5	468.4

Санамсаргүй модны өндөр ≈ O(lg n) тул анхны хувилбар дундажаар Θ(n lg n) бөгөөд Python-ы зүсэлт, `index` нь C хэл дээр хэрэгжсэн тул хурдан. Хоёр хувилбарын ялгаа бага, хэдхэн мс-ийн түвшинд хэмжилтийн хэлбэлзэл ч нөлөөлж байна. Сайжруулалтын жинхэнэ ач холбогдол хамгийн муу тохиолдолд харагдана.

Гинж (хамгийн муу тохиолдол):

n	Python анхны, ms	Python сайжр., ms	Java анхны, ms	Java сайжр., ms	Python анхны, KB	Python сайжр., KB	Java анхны, KB	Java сайжр., KB
500	3.535	0.629	0.322	0.046	2,211	241.6	1,020	52.9
1,000	14.5	1.346	0.973	0.054	8,436	595.5	3,992	111.7
2,000	64.7	2.762	6.794	0.100	32,604	1,303	15,797	229.3
4,000	269.4	6.314	53.7	0.289	127,815	2,719	62,844	464.4

7. Дүгнэлт

№	Бодлого	Дахин дуудах тэгшитгэл	Хугацаа	Нэмэлт санах ой
1	Majority Element	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \lg n)$	$\Theta(\lg n)$
2	Sorted Array → BST	$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$	$\Theta(n)$ гаралт + $\Theta(\lg n)$
3	Longest Nice Substring	$T(n) = \Sigma T(n_i) + \Theta(n)$, гүн ≤ 52	$\Theta(n)$	$\Theta(1)$ (Java)
4	Different Ways — анхны	$C(k) = 1 + 2\Sigma C(j) = 3^k$ дуудалт	3^k дахин бодолт	давхардсан жагсаалт
4	Different Ways — сайжруулсан	$(k+1)(k+2)/2$ дэд бодлого	$\Theta(C_k)$ (гаралттай ижил)	memo
5	Build Tree — анхны	$T(n) = T(k) + T(n-1-k) + \Theta(n)$	$\Theta(n \lg n)$ … $\Theta(n^2)$	$\Theta(n^2)$ хүртэл
5	Build Tree — сайжруулсан	$T(n) = T(k) + T(n-1-k) + \Theta(1)$	$\Theta(n)$	$\Theta(n)$

Гинжин модонд n хоёр дахин өсөхөд анхны хувилбарын санах ой яг 4 дахин (Java 15.8 → 62.8 MB), хугацаа Python-д 4.2 дахин (64.7 → 269 ms) өсөж, **$\Theta(n^2)$**-ийг баталж байна. Java-д хугацаа 7.9 дахин (6.8 → 53.7 ms) өссөн нь n^2 хэмжээтэй (62 MB) түр массивууд кэшэд багтахгүй, GC-ийн ачаалал нэмэгдсэнтэй холбоотой. Сайжруулсан хувилбарт хугацаа ≈ 2–3 дахин буюу **$\Theta(n)$**. n = 4000 үед хугацаа Python-д **43 дахин**, Java-д **186 дахин**, санах ой Python-д **47 дахин**, Java-д **135 дахин** багассан.

Гол дүгнэлтүүд:

1. **Хэсэгчлэх аргын өртөг нь хуваах, нэгтгэх алхмын өртгөөс шууд хамаарна.** 1, 2-р бодлого хоёулаа хоёр хагаст хуваадаг ч нэгтгэх алхам $\Theta(n)$ ба $\Theta(1)$ байгаагаас хариу нь $\Theta(n \lg n)$ ба $\Theta(n)$ болж ялгаатай гарсан.
2. **Дэд бодлого давхцаж байвал санах (memo).** 4-р бодлогод ижил дэд илэрхийллийг 3^k удаа бодож байсныг $(k+1)(k+2)/2$ болгов. Энэ нь динамик программчлалын санаа бөгөөд Лаб 4-ийн сэдэв.
3. **Хуулалт ба шугаман хайлтыг товъёг, хэш хүснэгтээр солих.** 5-р бодлогод хамгийн муу тохиолдлын $\Theta(n^2)$-ийг $\Theta(n)$ болгосон. Санамсаргүй өгөгдөл дээр ялгаа бага ч хамгийн муу оролтод хэдэн арав дахин.
4. **Хуваах стратеги хамгийн муу тохиолдлыг тодорхойлно.** 3-р бодлогод бүх муу үсгээр нэг дор хуваах нь гүнийг ≤ 52 болгож, $\Theta(n^2)$ болох муу тохиолдлоос сэргийлсэн.
5. **Python, Java хоёрын үр дүнгийн хандлага ижил, тогтмол үржүүлэгч өөр.** Java 15–160 дахин хурдан байсан ч (P4: 14×, P1: 89×, P2: 160×) n-ээр өсөх хурд хоёр хэлэнд ижил байв. Санах ойн хэмжилтэд «оргил» ба «нийт хуваарилалт» гэсэн хоёр өөр хэмжигдэхүүнийг ялгаж тайлбарлах хэрэгтэй.